

Chapter 4.5 - Pipelining Overview & Hazards

Comprehensive Revision Flashcards • 76 Key Concepts & Practice Questions

#1	Q1: What is pipelining in the context of computer architecture?
	Answer: It is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.
#2	Q2: What is the primary performance goal of pipelining?
	Answer: Pipelining aims to improve instruction throughput, which is the number of instructions completed per unit of time.
#3	Q3: What is the 'pipelining paradox' regarding instruction execution time?
	Answer: The time to complete a single instruction (latency) is not shorter, but the total time for many instructions is shorter due to increased throughput.
#4	Q4: Under ideal conditions with a large number of instructions, the speed-up from pipelining is approximately equal to what?
	Answer: The number of pipe stages.
#5	Q5: What are the five classic stages of a RISC-V instruction pipeline?
	Answer: 1. Fetch instruction, 2. Decode instruction/Read registers, 3. Execute, 4. Access memory, 5. Write back result.
#6	Q6: In a pipelined processor, what determines the duration of the clock cycle?
	Answer: The duration of the slowest pipeline stage.
#7	Q7: In a non-pipelined, single-cycle processor, what determines the duration of the clock cycle?
	Answer: The time required to execute the slowest possible instruction.
#8	Q8: What is a pipeline hazard?
	Answer: A situation in pipelining when the next instruction cannot execute in the following clock cycle for some reason.

#9	Q9: Events in a pipeline such as start-up and _____ affect performance when the number of tasks is not large compared to the number of stages.
	Answer: wind-down
#10	Q10: What is a structural hazard in a pipeline?
	Answer: A situation where a planned instruction cannot execute because the hardware cannot support the combination of instructions set to execute in the same clock cycle.
#11	Q11: What is a data hazard in a pipeline?
	Answer: A situation where an instruction cannot execute in the proper clock cycle because data it needs is not yet available from a previous instruction.
#12	Q12: What is a control hazard in a pipeline?
	Answer: A situation where the proper instruction cannot execute because the instruction that was fetched is not the one that is needed, typically due to a branch.
#13	Q13: What is another common name for a control hazard?
	Answer: A branch hazard.
#14	Q14: How does having a fixed length for all RISC-V instructions simplify pipelining?
	Answer: It makes it much easier to fetch instructions in the first pipeline stage and decode them in the second stage.
#15	Q15: How does RISC-V being a load-store architecture simplify pipelining?
	Answer: It allows the memory address calculation and memory access to occur in predictable, separate stages (EX and MEM).
#16	Q16: A specific type of data hazard where data from a `load` instruction is needed by a subsequent instruction before it's available is called a _____.
	Answer: load-use data hazard
#17	Q17: What is forwarding, also known as bypassing, used for?
	Answer: It is a method of resolving a data hazard by retrieving the missing data from internal buffers rather than waiting for it to be written to the register file.

#18	Q18: Why is forwarding sometimes insufficient, such as in a load-use data hazard?
	Answer: The desired data is not available until after the memory access stage, which is too late for the execution stage of the immediately following instruction.
#19	Q19: A stall initiated in order to resolve a hazard is often called a ____.
	Answer: bubble
#20	Q20: Besides hardware stalls, what is a software technique a compiler can use to mitigate data hazards?
	Answer: Reordering code to separate a dependent instruction from the one that produces its data.
#21	Q21: What is the simplest, albeit slow, hardware solution to a control hazard caused by a branch?
	Answer: To stall the pipeline after fetching a branch until the branch outcome and target address are known.
#22	Q22: A method of resolving a branch hazard by assuming an outcome and proceeding rather than waiting is known as ____.
	Answer: branch prediction
#23	Q23: What is a simple static branch prediction strategy mentioned in the text?
	Answer: To predict that conditional branches will always be untaken.
#24	Q24: What is dynamic branch prediction?
	Answer: A technique where hardware makes predictions based on the past behavior of each individual conditional branch, and may change predictions during program execution.
#25	Q25: What must the pipeline control do when a branch prediction is incorrect?
	Answer: It must ensure the wrongly fetched instructions have no effect and restart the pipeline from the correct branch address.
#26	Q26: An architectural solution to control hazards, used by MIPS, that always executes the instruction immediately following a branch is called a ____.
	Answer: delayed branch
#27	Q27: Pipelining improves instruction ____ rather than instruction ____.
	Answer: throughput, latency

#28	Q28: What is pipeline latency?
	Answer: The number of stages in a pipeline, representing the time it takes for a single instruction to complete its execution.
#29	Q29: In the laundry analogy, if the washer, dryer, folder, and storer each take 30 minutes, how long does pipelined laundry take for four loads?
	Answer: 3.5 hours.
#30	Q30: In the laundry analogy, if the washer, dryer, folder, and storer each take 30 minutes, how long does sequential laundry take for four loads?
	Answer: 8 hours.
#31	Q31: What are the five steps of instruction execution for RISC-V, which form the basis for its five-stage pipeline?
	Answer: 1. Fetch instruction, 2. Read registers/decode, 3. Execute/calculate address, 4. Access memory operand, 5. Write result to register.
#32	Q32: In the RISC-V single-cycle example, the clock cycle was 800 ps because it had to be stretched to accommodate the slowest instruction, which was ____.
	Answer: ld (load doubleword)
#33	Q33: In the graphical representation of forwarding in Figure 4.27, the path goes from the output of the EX stage of the `add` instruction to the input of the EX stage of which instruction?
	Answer: The `sub` instruction.
#34	Q34: In the code reordering example, which instruction was moved up to eliminate the pipeline stalls?
	Answer: The third `ld` instruction (`ld x4, 16(x31)`).
#35	Q35: Why is it impossible for a forwarding path to go from the MEM stage of one instruction to the EX stage of the *following* instruction without a stall?
	Answer: This would mean going backward in time, as the following instruction's EX stage occurs at the same time as the current instruction's MEM stage.
#36	Q36: Assuming conditional branches are 17% of instructions, what is the CPI if every branch causes a one-cycle stall and all other instructions have a CPI of 1?
	Answer: The CPI would be 1.17.

#37	Q37: In a pipeline diagram, what does shading on the right half of the register file symbol indicate?
	Answer: It indicates that the register file is being read in that stage.
#38	Q38: In a pipeline diagram, what does shading on the left half of the register file symbol indicate?
	Answer: It indicates that the register file is being written in that stage.
#39	Q39: The text states that modern implementations of the x86 architecture often translate complex x86 instructions into simpler, RISC-V-like operations before pipelining them. Why is this done?
	Answer: Because x86 instructions vary in length from 1 to 15 bytes, making direct pipelining considerably more challenging.
#40	Q40: In what pipeline stage does a RISC-V instruction write its result?
	Answer: The last stage (Write-back or WB).
#41	Q41: What is the primary performance metric improved by pipelining?
	Answer: Pipelining improves instruction throughput, which is the rate at which instructions are completed.
#42	Q42: What is the 'pipelining paradox'?
	Answer: The time to complete a single instruction (latency) is not shorter, but the total time for many instructions is shorter due to improved throughput.
#43	Q43: In the laundry analogy, what are the four sequential stages of a non-pipelined approach?
	Answer: 1. Wash, 2. Dry, 3. Fold, 4. Put away.
#44	Q44: In the laundry analogy, at what point are all pipeline stages operating concurrently?
	Answer: When the fourth load is in the washer, the third is in the dryer, the second is being folded, and the first is being put away.
#45	Q45: Under ideal conditions, the speed-up from pipelining is approximately equal to the ____.
	Answer: number of pipe stages
#46	Q46: Why is the speed-up in the laundry example (Figure 4.23) only 2.3 times instead of the potential 4 times?
	Answer: Because the pipeline is not completely full during the start-up and wind-down phases with only four loads of laundry.

#47	Q47: In a single-cycle implementation, what determines the length of the clock cycle?
	Answer: The clock cycle must be stretched to accommodate the slowest instruction.
#48	Q48: In a pipelined implementation, what determines the length of the clock cycle?
	Answer: The clock cycle must be long enough to accommodate the slowest pipeline stage.
#49	Q49: In the example from Figure 4.24, what was the single-cycle clock time and what was the pipelined clock time?
	Answer: The single-cycle time was 800 ps (for the slowest instruction, ld), and the pipelined time was 200 ps (for the slowest stage, memory access or ALU).
#50	Q50: Pipelining improves performance by increasing instruction _____, in contrast to decreasing the _____ of an individual instruction.
	Answer: throughput; execution time (or latency)
#51	Q51: What is one feature of the RISC-V instruction set that makes it easier to pipeline?
	Answer: All RISC-V instructions are the same length, which simplifies fetching and decoding.
#52	Q52: What is a second feature of the RISC-V instruction set design that aids pipelining?
	Answer: It has only a few instruction formats, with source/destination register fields in the same location.
#53	Q53: What is a third feature of the RISC-V instruction set that simplifies pipelining?
	Answer: Memory operands only appear in load or store instructions, allowing address calculation in one stage and memory access in the next.
#54	Q54: Term: Structural Hazard
	Answer: A hazard that occurs when the hardware cannot support the combination of instructions set to execute in the same clock cycle.
#55	Q55: What is a classic example of a structural hazard in a processor pipeline?
	Answer: When one instruction is accessing data memory while another instruction is being fetched from memory, if there is only a single memory unit.

#56	Q56: Term: Data Hazard
	Answer: A hazard that occurs when a planned instruction cannot execute because data it needs is not yet available.
#57	Q57: What causes a data hazard in a computer pipeline?
	Answer: It arises from the dependence of one instruction on an earlier one that is still in the pipeline.
#58	Q58: What is the primary hardware-based solution for resolving many data hazards?
	Answer: Forwarding (or bypassing), which retrieves the missing data from internal buffers rather than waiting for it to be written to a register.
#59	Q59: In the sequence `add x19, x0, x1` followed by `sub x2, x19, x3`, how does forwarding resolve the data hazard?
	Answer: The result from the ALU in the EX stage of the `add` instruction is forwarded directly to the ALU input for the EX stage of the `sub` instruction.
#60	Q60: A forwarding path is only valid if the destination stage is _____ in time than the source stage.
	Answer: later
#61	Q61: Term: Load-use data hazard
	Answer: A specific data hazard where data being loaded has not yet become available when needed by a subsequent instruction.
#62	Q62: Why can't forwarding alone resolve a load-use data hazard without a stall?
	Answer: The desired data is only available after the memory access (MEM) stage, which is too late for the execution (EX) stage of the immediately following instruction.
#63	Q63: What is a pipeline stall, also known as a bubble?
	Answer: A stall initiated in order to resolve a hazard, effectively inserting a delay into the pipeline.
#64	Q64: In the example `a = b + e; c = b + f;`, how can the code be reordered to avoid pipeline stalls?
	Answer: By moving the load for `f` up, between the loads for `b` and `e` and the first `add` instruction, to separate the dependent instructions.

#65	Q65: Term: Control Hazard
	Answer: Also called a branch hazard, it occurs when the proper instruction cannot execute because the instruction that was fetched is not the one that is needed.
#66	Q66: What is the simplest, though slow, solution to a control hazard caused by a conditional branch?
	Answer: To stall the pipeline after fetching a branch until the outcome is determined.
#67	Q67: Assuming a 1-cycle stall for every conditional branch, what would be the impact on CPI if branches are 17% of instructions?
	Answer: The CPI would increase from an ideal 1.0 to 1.17, representing a 17% slowdown.
#68	Q68: What is branch prediction?
	Answer: A method of resolving a branch hazard that assumes a given outcome and proceeds from that assumption rather than waiting.
#69	Q69: What happens in a 'predict-not-taken' scheme if the branch is actually taken?
	Answer: The pipeline stalls, and the incorrectly fetched instructions are flushed while the pipeline restarts from the correct branch address.
#70	Q70: What is the key difference between static and dynamic branch prediction?
	Answer: Static prediction is rigid (e.g., always predict taken for backward branches), while dynamic prediction uses the past behavior of each specific branch to make its guess.
#71	Q71: What is the typical accuracy of modern dynamic branch predictors?
	Answer: They can correctly predict conditional branches with more than 90% accuracy.
#72	Q72: What is a 'delayed branch'?
	Answer: A solution to control hazards where the instruction immediately following the branch is always executed, regardless of the branch outcome.
#73	Q73: For the code sequence: `ld x10, 0(x10)` followed by `add x11, x10, x10`, can it execute without stalling?
	Answer: No, this is a load-use hazard that must stall for one cycle, even with forwarding.

#74	Q74: For the code sequence: `add x11, x10, x10` followed by `addi x12, x11, 5`, can it execute without stalling?
	Answer: Yes, this data hazard can be resolved using only forwarding.
#75	Q75: For the code sequence: `addi x11, x10, 1` followed by `addi x12, x10, 2` and `addi x13, x10, 3`, can it execute without issues?
	Answer: Yes, it can execute without stalling or forwarding because there are no data dependencies between the instructions.
#76	Q76: What is the definition of pipeline latency?
	Answer: The number of stages in a pipeline or the number of stages between two instructions during execution.